



南开大学
Nankai University

南 开 大 学

计算机网络实验报告

实验 3：配置 Web 服务器，捕获 HTTP 报文并分析

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：徐敬东、张建忠

2025 年 12 月 31 日

目录

1 实验要求	1
2 实验原理	1
2.1 HTTP 协议概述	1
2.2 TCP/IP 分层封装机制	1
2.3 Web 服务器工作流程	2
3 实验过程	2
3.1 Web 服务器搭建	2
3.1.1 使用 Python 启动 HTTP 服务器	2
3.1.2 端口设置与访问测试	3
3.1.3 服务器日志解读	3
3.2 Web 页面设计与实现	3
3.2.1 HTML 结构设计	3
3.2.2 HTML 源码展示	4
3.2.3 页面展示	6
4 实验结果分析	6
4.1 Wireshark 抓包与分析	6
4.1.1 抓包配置	6
4.1.2 报文捕获过程记录	7
4.2 HTTP 报文详细分析	7
4.2.1 HTTP 请求报文分析 (GET /)	7
4.2.2 HTTP 响应报文分析 (200 OK)	9
4.3 HTTP 完整交互流程分析	10
4.3.1 TCP 三次握手建立连接	10
4.3.2 请求-响应循环与长连接	11
4.3.3 TCP 四次挥手释放连接	11
5 实验总结	11
5.1 实验遇到的问题与解决方案	11
5.2 实验心得体会	12

1 实验要求

1. 搭建 Web 服务器（自由选择系统），并制作简单的 Web 页面，包含简单文本信息（包含专业、学号、姓名）和 6 幅图像。
2. 通过浏览器获取自己编写的 Web 页面，使用 Wireshark 捕获浏览器与 Web 服务器的交互过程。
3. 分析两个报文的封装层次，包括数据链路层、互连层、传输层、应用层。
4. 对整个 HTTP 交互过程进行详细说明，使用 Wireshark 过滤器使其只显示 HTTP 协议。
5. 提交 HTML 文档、Wireshark 捕获文件和实验报告。

注：页面不要太复杂，包含所要求的基本信息即可。使用 HTTP，不使用 HTTPS。

2 实验原理

2.1 HTTP 协议概述

HTTP (HyperText Transfer Protocol, 超文本传输协议) 是万维网数据通信的基础, 属于应用层协议, 基于 Client/Server (B/S) 模型工作。它定义了客户端 (浏览器) 与 Web 服务器之间交换数据的格式和规则。本实验涉及的 HTTP/1.1 版本主要具有以下特点:

- **无状态性 (Stateless)**: 服务器不会在两个请求之间保留任何数据状态, 每个请求都是独立的。
- **请求/响应模型**: 通信由客户端发起请求 (Request), 服务器返回响应 (Response)。
- **持久连接 (Persistent Connection)**: HTTP/1.1 默认开启 Keep-Alive, 允许在同一个 TCP 连接上发送多个 HTTP 请求 (如加载网页后的多张图片), 显著减少了建立连接的延迟。(注: 虽然现代浏览器默认使用 HTTP/1.1, 但本实验所使用的 Python http.server 模块默认响应为 HTTP/1.0, 通常不具备长连接特性)

2.2 TCP/IP 分层封装机制

在网络传输中, 数据自上而下经过四层封装, 每一层都在前一层的基础上增加控制首部 (Header)。针对本机环回测试环境, 其封装过程如下:

- **应用层 (Application Layer)**: 生成 HTTP 报文, 包含请求方法 (GET)、URL、协议版本及头部字段。
- **传输层 (Transport Layer)**: TCP 协议为数据添加源端口和目的端口 (Web 服务默认为 80), 并提供序列号与确认号, 确保数据的可靠、有序传输。
- **网络层 (Internet Layer)**: IP 协议封装源 IP (127.0.0.1) 和目的 IP (127.0.0.1), 负责逻辑寻址。
- **数据链路层 (Link Layer)**: 由于是在本机内部通信, 数据不经过物理网卡, 因此不使用以太网帧格式 (无 MAC 地址), 而是采用 Null/Loopback 封装格式, 仅包含一个协议族标识字段 (Family ID: IP)。

2.3 Web 服务器工作流程

Web 服务器（如本实验使用的 Python http.server）本质上是一个运行在应用层的守护进程，其核心工作流程包括：

- **监听 (Listening)**：启动后在指定端口（80）等待连接请求。
- **建立连接 (Connection)**：通过 TCP 三次握手与客户端建立可靠通道。
- **处理请求 (Processing)**：解析客户端发来的 HTTP 请求报文，根据 URL 路径在本地文件系统（如 Network_Lab3 目录）中定位资源文件（如 index.html 或.jpg 图片）。
- **发送响应 (Response)**：将读取的资源内容封装在 HTTP 响应报文中发送回客户端。若开启长连接，则保持 TCP 通道不关闭，等待后续请求；否则通过四次挥手释放连接。

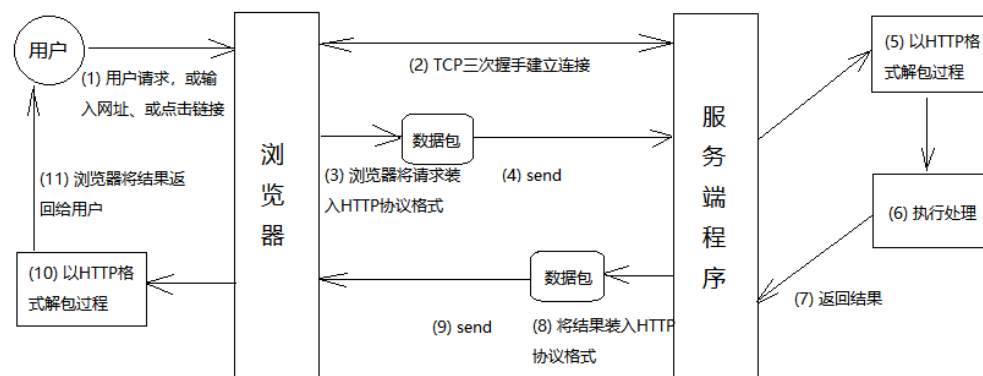


图 1: Web 服务器交互流程原理图

3 实验过程

3.1 Web 服务器搭建

3.1.1 使用 Python 启动 HTTP 服务器

本实验使用 Python 标准库中内置的 http.server 模块来搭建轻量级 Web 服务器。该方案无需繁琐的安装过程，具备跨平台特性，且能实时在控制台输出请求日志，非常适合开发调试与协议分析实验。

实验步骤：

1. **建立项目目录**：在本地磁盘建立项目根目录 Network_Lab3，将编写好的 index.html 及图片资源文件夹置于该目录下。
2. **切换工作目录**：打开命令提示符（CMD）或 PowerShell，使用 cd 命令切换至该目录。
3. **启动服务器**：输入启动命令：

```
python -m http.server 80
```

系统提示 Serving HTTP on :: port 80 (http://[::]:80/), 表明服务器已成功监听 80 端口，等待客户端连接。

```
PS K:\大三上\计算机网络\实验\Network_Lab3> python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```

图 2: 等待客户端连接

3.1.2 端口设置与访问测试

端口选择 : 选用 HTTP 协议默认的 80 端口。这使得在浏览器访问时无需显式输入端口号（即输入 `http://127.0.0.1` 即可，而非 `http://127.0.0.1:8000`），模拟了真实的 Web 访问体验。

访问测试 : 打开 Microsoft Edge 浏览器，在地址栏输入本机环回地址 `127.0.0.1`。浏览器成功渲染出包含个人信息及图片展示的实验页面，证明服务器搭建成功。

3.1.3 服务器日志解读

服务器启动后，控制台会实时打印交互日志，如下所示：

```
127.0.0.1 - - [31/Dec/2025 10:00:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Dec/2025 10:00:05] "GET /image/1.jpg HTTP/1.1" 200 -
```

```
PS K:\大三上\计算机网络\实验\Network_Lab3> python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET / HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/1.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/2.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/3.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/5.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/6.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:03] "GET /image/4.jpg HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:04] code 404, message File not found
::ffff:127.0.0.1 - - [31/Dec/2025 10:07:04] "GET /favicon.ico HTTP/1.1" 404 -
```

图 3: 交互日志

日志分析 : 每一条日志包含了客户端 IP (`127.0.0.1`)、访问时间、请求方法 (`GET`)、请求路径 (/ 即主页) 以及 HTTP 状态码 (`200` 表示成功)。通过观察日志，可以直观地看到浏览器在加载主页后，自动并发起了对后续图片资源的请求。

3.2 Web 页面设计与实现

3.2.1 HTML 结构设计

页面采用 HTML5 语义化标签构建，结构清晰分层：

- `<header>`: 页面标题区域
- `<section class="student-card">`: 学生信息展示区
- `<section class="gallery-section">`: 图片展示区
- `<footer>`: 页脚信息

3.2.2 HTML 源码展示

Listing 1: 完整的 HTML 源码

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>计算机网络实验 – Web服务器配置</title>
7   <style>
8     /* CSS 样式重置与布局优化 */
9     body { font-family: 'Segoe UI', 'Microsoft YaHei', sans-serif; line-
10       height: 1.6; margin: 0; background-color: #f0f2f5; color: #333; }
11     .container { max-width: 1000px; margin: 30px auto; background: white;
12       padding: 40px; border-radius: 12px; box-shadow: 0 4px 20px rgba
13       (0,0,0,0.08); }
14     header { border-bottom: 3px solid #1a73e8; padding-bottom: 20px;
15       margin-bottom: 30px; text-align: center; }
16     h1 { margin: 0; color: #1a73e8; font-size: 2.2em; }
17     .subtitle { color: #666; margin-top: 10px; }
18     /* 个人信息卡片设计 – 已修改对齐方式 */
19     .student-card { background: #e8f0fe; border-left: 5px solid #1a73e8;
20       padding: 20px; border-radius: 4px; margin-bottom: 30px; }
21     .info-row {
22       display: grid;                /* 使用 Grid 布局 */
23       grid-template-columns: 1fr 1fr; /* 左右各占 50%, 建立中轴线 */
24       width: 60%;                  /* 限制整体宽度, 让内容紧凑一些 */
25       margin: 10px auto;           /* 让整行在卡片中居中 */
26       font-size: 1.1em;
27       border-bottom: 1px dashed #cce0ff; /* 虚线分割 */
28       padding-bottom: 5px;
29     }
30     .info-label {
31       text-align: right;             /* 标签文字靠右对齐 */
32       padding-right: 30px;          /* 标签与中轴线的间距 */
33       font-weight: bold;
34       color: #555;
35     }
36     .info-val {
37       text-align: left;             /* 内容文字靠左对齐 */
38       padding-left: 30px;           /* 内容与中轴线的间距 */
39     }
40     /* 图片网格布局 */
41     .gallery { display: grid; grid-template-columns: repeat(auto-fit,
42       minmax(280px, 1fr)); gap: 25px; }
43     .photo-card { background: #fff; border: 1px solid #eee; border-radius
44       : 8px; overflow: hidden; transition: transform 0.3s ease, box-
45       shadow 0.3s ease; }
```

```

38     .photo-card:hover { transform: translateY(-5px); box-shadow: 0 10px
      20px rgba(0,0,0,0.1); }
39     .photo-card img { width: 100%; height: 200px; object-fit: cover;
      display: block; }
40     .caption { padding: 15px; text-align: center; font-size: 0.9em; color
      : #666; background: #fafafa; border-top: 1px solid #eee; }
41     footer { text-align: center; margin-top: 50px; font-size: 0.8em;
      color: #999; }
42     </style>
43 </head>
44 <body>
45     <div class="container">
46         <header>
47             <h1>Web 服务器配置与 HTTP 分析</h1>
48             <p class="subtitle">计算机网络实验三</p>
49         </header>
50         <section class="student-card">
51             <div class="info-row"><span class="info-label">专业: </span>
52             <span class="info-val">物联网工程</span></div>
53             <div class="info-row"><span class="info-label">学号: </span>
54             <span class="info-val">2310422</span></div>
55             <div class="info-row"><span class="info-label">姓名: </span>
56             <span class="info-val">谢小珂</span></div>
57         </section>
58         <section class="gallery-section">
59             <h2 style="border-left: 4px solid #1a73e8; padding-left: 10px;">
      实验图片展示</h2>
60             <p style="margin-bottom: 20px; color: #666;">以下是通过本地 Web
      服务器加载的 6 幅实验图像: </p>
61             <div class="gallery">
62                 <div class="photo-card">
63                 <div class="caption">图 1 </div></div>
64                 <div class="photo-card">
65                 <div class="caption">图 2 </div></div>
66                 <div class="photo-card">
67                 <div class="caption">图 3 </div></div>
68                 <div class="photo-card">
69                 <div class="caption">图 4 </div></div>
70                 <div class="photo-card">
71                 <div class="caption">图 5 </div></div>
72                 <div class="photo-card">
73                 <div class="caption">图 6 </div></div>
74             </div>
75         </section>
76         <footer>
77             <p>© 2025 谢小珂 | 物联网工程 | 计算机网络实验</p>
78         </footer>
79     </div>

```

80

</body>

81

</html>

3.2.3 页面展示

启动服务器后，在浏览器中访问页面，最终效果如图4所示。页面采用”科技蓝”配色方案(#1a73e8)，整体风格简洁现代，完整展示了专业、学号、姓名等个人信息及 6 幅实验图像。

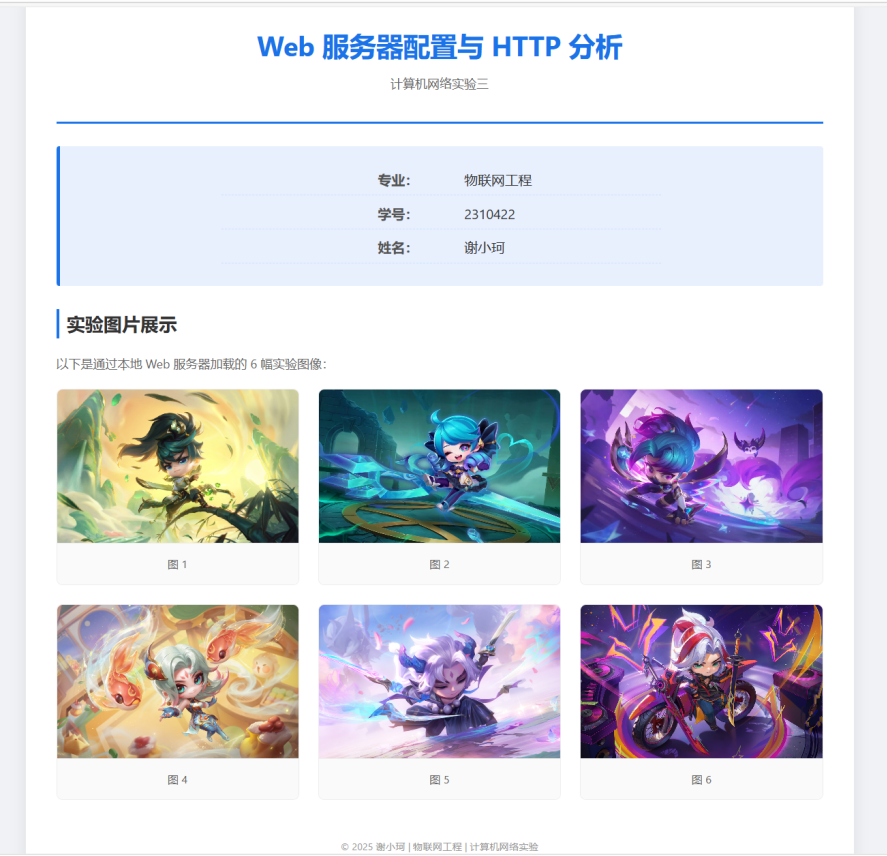


图 4: web 页面展示效果

4 实验结果分析

4.1 Wireshark 抓包与分析

4.1.1 抓包配置

为了准确分析 Web 服务器与浏览器之间的底层交互，本实验采用 Wireshark 网络协议分析工具。

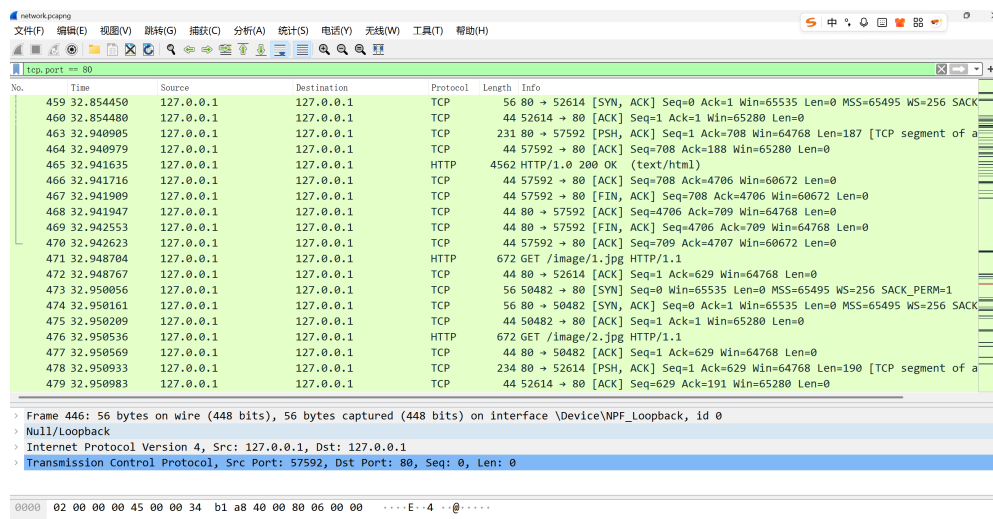
接口选择: 由于服务器运行在本地(Localhost),流量不经过物理网卡,因此必须选择”Adapter for loopback traffic capture” 接口。

过滤器设置: 为了完整观察从连接建立到断开的全过程，初始过滤器设置为 tcp.port == 80。

4.1.2 报文捕获过程记录

1. 启动 Wireshark 捕获。
2. 使用浏览器（无痕模式）访问 `http://127.0.0.1`，待页面及图片资源加载完成后，立即关闭标签页以触发连接释放。
3. 停止捕获。

捕获结果清晰地展示了从 TCP 三次握手建立连接，到 HTTP 请求与响应传输，再到 TCP 四次挥手断开连接的完整生命周期，如图5所示。



No.	Time	Source	Destination	Protocol	Length	Info
459	32.854450	127.0.0.1	127.0.0.1	TCP	56	80 → 52614 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
460	32.854480	127.0.0.1	127.0.0.1	TCP	44	52614 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
463	32.940905	127.0.0.1	127.0.0.1	TCP	231	80 → 57592 [PSH, ACK] Seq=1 Ack=708 Win=64768 Len=187 [TCP segment of a ...]
464	32.940979	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=188 Win=65280 Len=0
465	32.941635	127.0.0.1	127.0.0.1	HTTP	4562	HTTP/1.0 200 OK (text/html)
466	32.941716	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=4706 Win=60672 Len=0
467	32.941909	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [FIN, ACK] Seq=708 Ack=4706 Win=60672 Len=0
468	32.941947	127.0.0.1	127.0.0.1	TCP	44	80 → 57592 [ACK] Seq=4706 Ack=709 Win=64768 Len=0
469	32.942553	127.0.0.1	127.0.0.1	TCP	44	80 → 57592 [FIN, ACK] Seq=4706 Ack=709 Win=64768 Len=0
470	32.942623	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=709 Ack=4707 Win=60672 Len=0
471	32.948704	127.0.0.1	127.0.0.1	HTTP	672	GET /image/1.jpg HTTP/1.1
472	32.948767	127.0.0.1	127.0.0.1	TCP	44	80 → 52614 [ACK] Seq=1 Ack=629 Win=64768 Len=0
473	32.950056	127.0.0.1	127.0.0.1	TCP	56	50482 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
474	32.950161	127.0.0.1	127.0.0.1	TCP	56	80 → 50482 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
475	32.950209	127.0.0.1	127.0.0.1	TCP	44	50482 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
476	32.950536	127.0.0.1	127.0.0.1	HTTP	672	GET /image/2.jpg HTTP/1.1
477	32.950569	127.0.0.1	127.0.0.1	TCP	44	80 → 50482 [ACK] Seq=1 Ack=629 Win=64768 Len=0
478	32.950933	127.0.0.1	127.0.0.1	TCP	234	80 → 52614 [PSH, ACK] Seq=1 Ack=629 Win=64768 Len=190 [TCP segment of a ...]
479	32.950983	127.0.0.1	127.0.0.1	TCP	44	52614 → 80 [ACK] Seq=629 Ack=191 Win=65280 Len=0

图 5: Wireshark 捕获的完整交互过程

4.2 HTTP 报文详细分析

本节选取一次完整的 HTTP 交互（请求主页 `index.html`）进行深层封装分析，验证 TCP/IP 协议栈的分层结构。

4.2.1 HTTP 请求报文分析 (GET /)

分析对象：Frame 449，该报文是浏览器向服务器发起的资源请求。

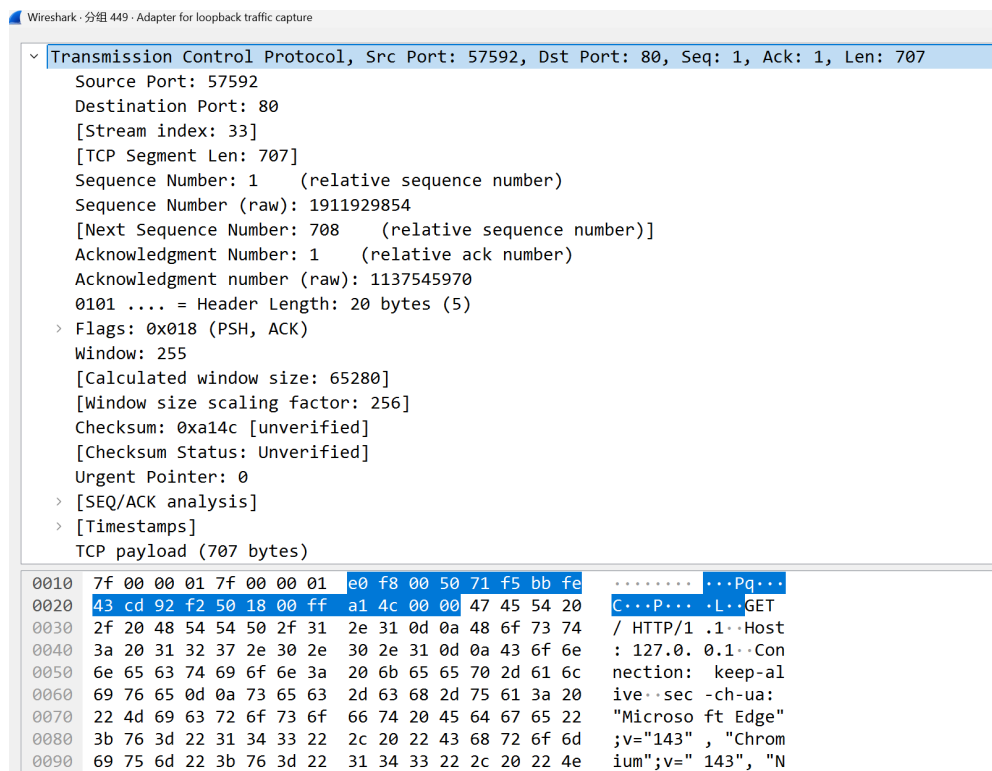


图 6: HTTP 请求报文详情 (展开 TCP 和 HTTP 层)

数据链路层 (Data Link Layer)

- **封装格式:** Null/Loopback。
- **分析:** 由于通信双方均为本机 (127.0.0.1)，数据包直接经过操作系统的回环接口转发。与以太网不同，Loopback 帧头部非常简洁，不包含 MAC 地址。Family: IP (2) 字段标识了上层协议为 IPv4。

网络层 (Network Layer)

- **协议:** IPv4。
- **地址:** 源 IP (Src) 与目的 IP (Dst) 均为 127.0.0.1。
- **关键字段:** Protocol: TCP (6)，表明上层使用可靠传输协议。

传输层 (Transport Layer)

- **端口信息:** 源端口为 57592 (浏览器随机分配)，目的端口为 80 (Web 服务器监听端口)。
- **标志位:** [PSH, ACK]。在传输层中，观察到 Flags 字段为 0x018 (PSH, ACK)。表示发送方 (浏览器) 提示接收方 (服务器 TCP 栈) 应立即将该数据段推送给应用层进程，而不是在缓冲区排队等待填满。这在交互式 HTTP 请求中非常常见，确保了服务器能第一时间处理 GET 请求。

应用层 (Application Layer)

- **请求行:** GET / HTTP/1.1。
- **Host:** 127.0.0.1。HTTP/1.1 协议强制要求的头部，用于指定目标主机。
- **User-Agent:** 标识客户端身份 (如 Microsoft Edge/Chrome)，服务器可据此返回适配的页面。

- **连接复用机制**：在应用层头部中，字段 Connection: keep-alive 至关重要。它告知服务器在发送响应后不要立即关闭 TCP 连接（即不要发送 FIN 包）。这使得后续的 6 张图片请求（Frame 471, 476 等）可以直接复用当前的 TCP Stream (Stream index: 33)，从而避免了为每张图片重复进行三次握手，显著降低了网络延迟。

4.2.2 HTTP 响应报文分析 (200 OK)

分析对象：Frame 465，该报文是服务器返回的 HTML 文件数据。

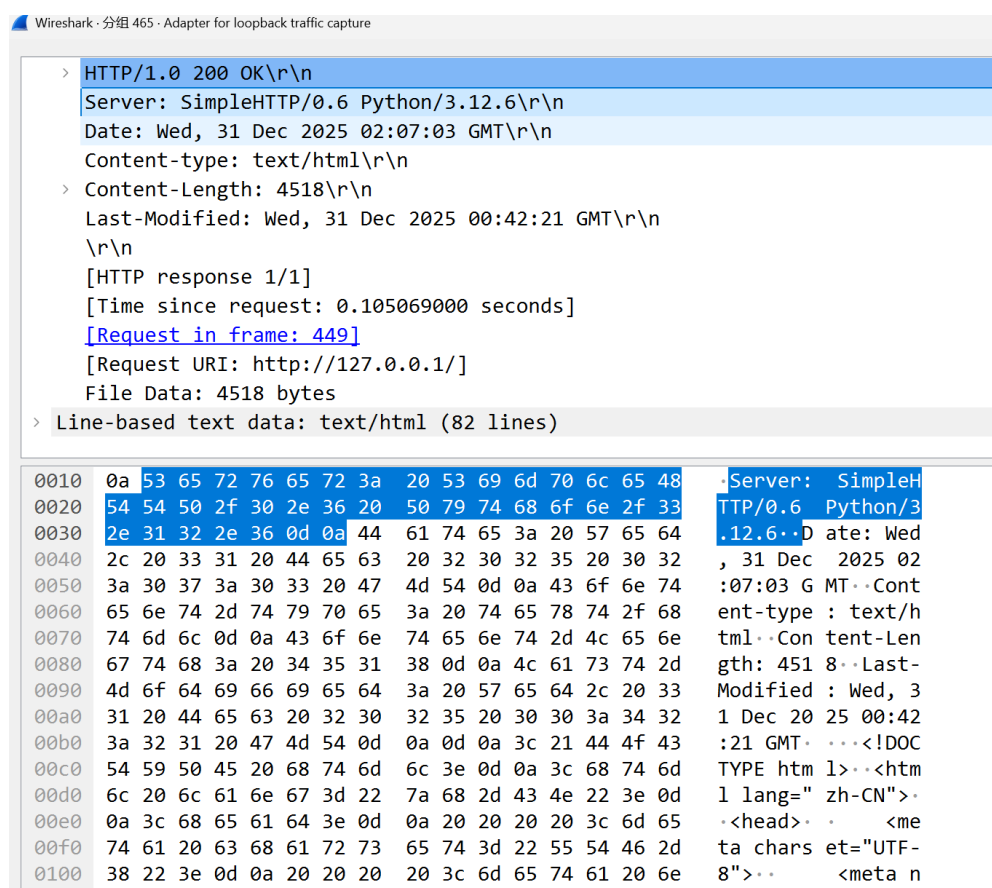


图 7: HTTP 响应报文详情（展开 TCP 和 HTTP 层）

传输层分析

- **可靠性确认**：该报文的确认号（Ack）等于请求报文的序列号（Seq）加数据长度，体现了 TCP 的可靠传输机制。
- **数据重组**：由于响应内容（HTML 代码）较大，Wireshark 显示 [Reassembled PDU]，说明应用层数据被拆分成了多个 TCP 段进行传输，并在接收端完成了重组。

应用层分析

- **状态行**：HTTP/1.0 200 OK。状态码 200 表示请求成功，资源存在。（注意此处服务器返回的版本为 HTTP/1.0，而非浏览器请求的 1.1。这表明 Python 简易服务器仅支持基础的 HTTP/1.0 协议，通常会在响应结束后立即断开连接）。
- **Server**：SimpleHTTP/0.6 Python/3.12.6。明确标识了后端服务器使用的是 Python 内置的轻量级 HTTP 服务模块。这不仅验证了实验环境（Python 3.12.6），在网络安全视角下，

这也是一种“信息泄露”。在实际生产环境中，通常会隐藏具体的版本号（如仅显示 Server: Apache）以防止针对特定版本的漏洞攻击。

- **Content-Type:** text/html。告知浏览器响应体中的数据是 HTML 文档，需进行网页渲染。
- **Content-Length:** 4518。指示了响应实体的精确字节数。
- **Last-Modified:** Wed, 31 Dec 2025 00:42:21 GMT：当浏览器下次访问同一页面时，会在请求头中携带 If-Modified-Since 字段发送该时间。如果文件未修改，服务器将仅返回 304 Not Modified（不传输实体内容），从而节省带宽。

4.3 HTTP 完整交互流程分析

4.3.1 TCP 三次握手建立连接

在 HTTP 请求发出前，观察到前三个报文完成了连接建立，验证了 HTTP 是基于 TCP 的应用层协议。

No.	Time	Source	Destination	Protocol	Length	Info
446	32.825115	127.0.0.1	127.0.0.1	TCP	56	57592 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
447	32.825233	127.0.0.1	127.0.0.1	TCP	56	80 → 57592 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
448	32.825274	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
449	32.836566	127.0.0.1	127.0.0.1	HTTP	751	GET / HTTP/1.1
450	32.836612	127.0.0.1	127.0.0.1	TCP	44	80 → 57592 [ACK] Seq=1 Ack=708 Win=64768 Len=0
458	32.854375	127.0.0.1	127.0.0.1	TCP	56	52614 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
459	32.854450	127.0.0.1	127.0.0.1	TCP	56	80 → 52614 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
460	32.854480	127.0.0.1	127.0.0.1	TCP	44	52614 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
463	32.940905	127.0.0.1	127.0.0.1	TCP	231	80 → 57592 [PSH, ACK] Seq=1 Ack=708 Win=64768 Len=187 [TCP segment of a reassembled data segment]
464	32.940979	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=188 Win=65280 Len=0
465	32.941635	127.0.0.1	127.0.0.1	HTTP	4562	HTTP/1.0 200 OK (text/html)
466	32.941716	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=4706 Win=60672 Len=0
467	32.941909	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [FIN, ACK] Seq=708 Ack=4706 Win=60672 Len=0

图 8: TCP 三次握手过程 (SYN, SYN-ACK, ACK)

- **第一次握手 [SYN]**: 客户端发送 SYN 包 (Seq=0)，请求建立连接。
- **第二次握手 [SYN, ACK]**: 服务器回复 SYN+ACK 包 (Ack=1)，确认收到请求并同意连接。
- **第三次握手 [ACK]**: 客户端发送 ACK 包，连接状态转为 Established。

在分析 TCP 三次握手过程时，特别关注了第一次握手 (Frame 446) 报文中的 TCP 选项字段。通常，在标准以太网环境中，MSS (Maximum Segment Size, 最大报文段长度) 为 1460 字节 (对应 MTU 1500 字节减去 IP 和 TCP 头部各 20 字节)。然而，捕获到的 SYN 包中显示 MSS=65495。这是由于实验在本地回环 (Loopback) 接口上进行，数据不经过物理网卡，操作系统的环回接口采用最大 MTU (65535 字节)，减去 40 字节的 IP 与 TCP 头部后，得到此超大 MSS 值。这一细节确凿地证明了 Wireshark 捕获到的流量为未经物理层分片的本地主机内部通信，清晰地体现了实验环境的特性。相关细节如图8所示。

4.3.2 请求-响应循环与长连接

连接建立后，浏览器发送 GET /请求，服务器返回 index.html。

- **并发加载与复用**：浏览器解析 HTML 后，发现页面包含 6 张图片，随即发起 6 个 GET /image/x.jpg 请求。
- **HTTP/1.0 短连接行为**：尽管浏览器在请求头中申请了 Connection: keep-alive，但在收到服务器的 HTTP/1.0 响应后，浏览器（客户端）在 Frame 467 立即发送了 FIN 包主动断开连接，随后才发起新的连接请求图片。这表明双方未能成功协商长连接，导致了连接的重建。
- **重新握手**：当浏览器请求后续图片资源时（如 Frame 471），由于之前的连接已断开，不得不发起新的 TCP 三次握手（见图 5 中的 Frame 473，新的 SYN 包）。这证明本实验使用的 Python 简易服务器遵循 HTTP/1.0 标准，未开启长连接复用，每个资源的传输都需要独立的 TCP 连接周期。

4.3.3 TCP 四次挥手释放连接

实验结束关闭页面时，捕获到了完整的连接释放过程。

Time	Source	Destination	Protocol	Length	Info
459 32.854450	127.0.0.1	127.0.0.1	TCP	56	80 → 52614 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
460 32.854480	127.0.0.1	127.0.0.1	TCP	44	52614 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
463 32.940905	127.0.0.1	127.0.0.1	TCP	231	80 → 57592 [PSH, ACK] Seq=1 Ack=708 Win=64768 Len=187 [TCP segment of a reassembled data segment]
464 32.940979	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=188 Win=65280 Len=0
465 32.941635	127.0.0.1	127.0.0.1	HTTP	4562	HTTP/1.0 200 OK (text/html)
466 32.941716	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=708 Ack=4706 Win=60672 Len=0
467 32.941909	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [FIN, ACK] Seq=708 Ack=4706 Win=60672 Len=0
468 32.941947	127.0.0.1	127.0.0.1	TCP	44	80 → 57592 [ACK] Seq=4706 Ack=709 Win=64768 Len=0
469 32.942553	127.0.0.1	127.0.0.1	TCP	44	80 → 57592 [FIN, ACK] Seq=4706 Ack=709 Win=64768 Len=0
470 32.942623	127.0.0.1	127.0.0.1	TCP	44	57592 → 80 [ACK] Seq=709 Ack=4707 Win=60672 Len=0
471 32.948704	127.0.0.1	127.0.0.1	HTTP	672	GET /image/1.jpg HTTP/1.1
472 32.948767	127.0.0.1	127.0.0.1	TCP	44	80 → 52614 [ACK] Seq=1 Ack=629 Win=64768 Len=0
473 32.950056	127.0.0.1	127.0.0.1	TCP	56	50482 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1

图 9: TCP 四次挥手过程 (FIN-ACK, ACK)

- **主动关闭**：浏览器（端口 57592）发送 [FIN, ACK] (Frame 467)，表示数据接收完毕，请求断开连接。
- **被动关闭**：服务器（端口 80）回复 [ACK] (Frame 468)，确认收到关闭请求。
- **连接终结**：随后服务器也发送 [FIN, ACK] (Frame 469)，浏览器回复 ACK (Frame 470)，双方完成交互，连接彻底关闭。

5 实验总结

5.1 实验遇到的问题与解决方案

TCP 握手/挥手包丢失

- **问题**：过滤器设为 http 时看不到 SYN/FIN 包；刷新页面也未触发新连接。

- **解决：**将过滤器改为 `tcp.port == 80`；使用浏览器无痕模式访问并立即关闭标签页，强制触发完整的连接建立与释放过程。

抓包接口选择错误

- **问题：**选择 WLAN/Ethernet 无法抓取本地 localhost 流量。
- **解决：**切换为“Adapter for loopback traffic capture”接口，成功捕获本地回环流量。

5.2 实验心得体会

理论与实践结合 通过 Wireshark 逐层查看 Frame → Loopback → TCP → HTTP 的封装结构，让书本上的协议分层理论变得直观可见。从数据链路层的 Loopback 封装，到网络层的 IP 地址，再到传输层的端口与序列号，最后到应用层的 HTTP 报文，每一层的封装都清晰展示了 TCP/IP 协议栈的实际工作方式。

深入理解协议机制 在排错过程中，深刻理解了 HTTP/1.0 短连接与 HTTP/1.1 长连接的区别。在实验中，我观察到虽然浏览器请求 Keep-Alive，但受限于服务器版本，连接被频繁断开和重建（多次握手）。这让我反向理解了长连接的重要性——如果支持 Keep-Alive，就能避免这些重复的握手开销，从而提升性能。

全栈视角提升 从 Python 服务器搭建、HTML/CSS 页面设计到底层抓包分析，打通了从应用层到底层的完整技术链路，锻炼了综合工程能力。实验不仅加深了对计算机网络理论的理解，还提升了实际动手能力，为后续学习更复杂的网络协议和应用开发打下了坚实基础。